ÉCOLE POLYTECHNIQUE MONTRÉAL

# Solution exercices pour la préparation de l'examen final : bloc matériel INF3610 Systèmes Embarqués

## Chapitre 3

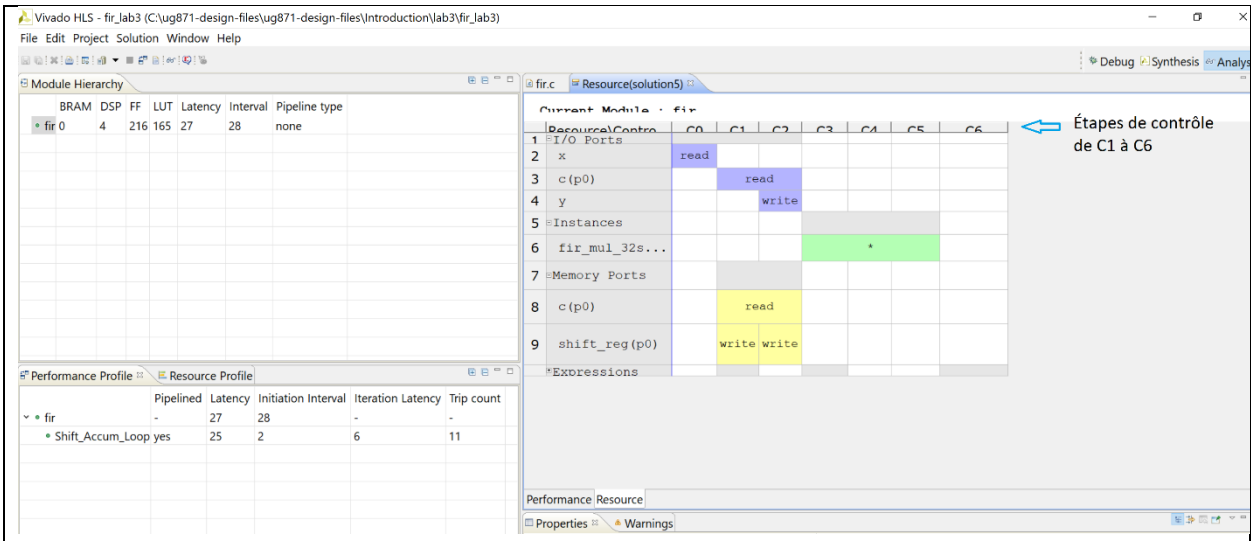
## Question 1 Optimisation pour synthèse HLS

La figure 1.1 représente le code du FIR de dimension 11 similaire à celui présenté en classe.

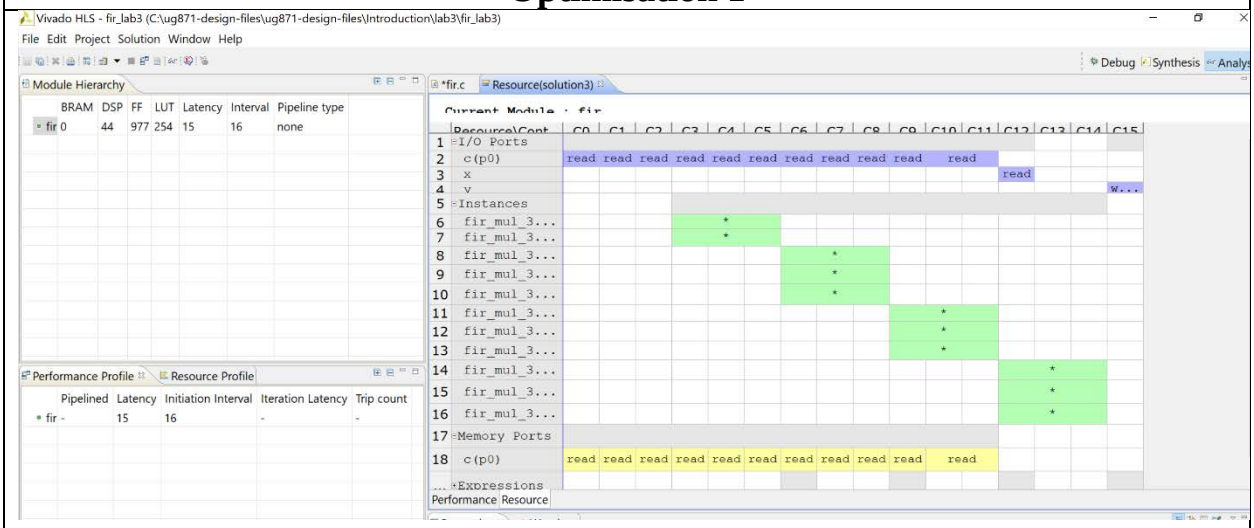
```
#include "fir.h"

void fir (
  data_t *y,
  coef_t c[N],
  data_t x
  ) {
#pragma HLS INTERFACE ap_vld port=y  //ajout d'un protocole handshacking pour
#pragma HLS INTERFACE ap_vld port=x  //synchorniser les données d'entrée
#pragma HLS RESOURCE variable=c core=RAM_1P_BRAM


  static data_t shift_reg[N];
  acc_t acc;
  data_t data;
  int i;

  acc=0;
  Shift_Accum_Loop: for (i=N-1;i>=0;i--) {
    if (i==0) {
            shift_reg[0]=x;
            data = x;
    } else {
            shift_reg[i]=shift_reg[i-1];
            data = shift_reg[i];
    }
    acc+=data*c[i];;
  }
  *y=acc;
}
```

**Figure 1.1**

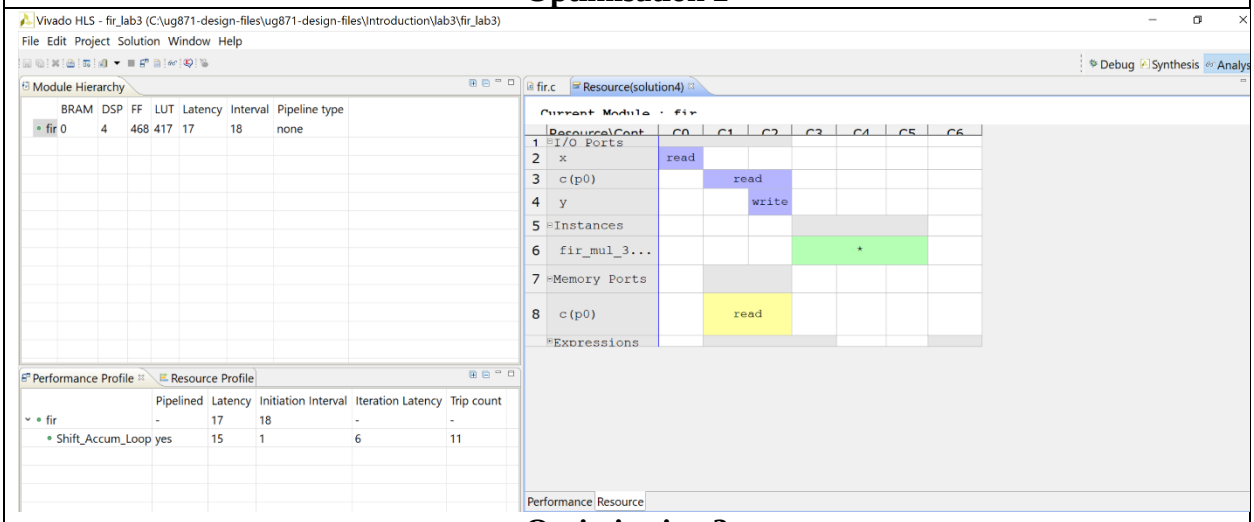
On décide d'optimiser ce code dans HLS Vivado. Pour chacune des 3 optimisations de la page suivante, indiquez-la (les) directive(s) (pragma) qui a (ont) été utilisé(es). Justifiez bien votre réponse. Voir l'annexe au besoin.




**Optimisation 1**




**Optimisation 2**




**Optimisation 3**

*Optimisation 1 : Pragma HLS pipeline – on a 11 itérations, une latence de 6, un II = 2 (à cause du 2 cycles du shif). Ça veut donc dire 6 + 2 * 11 = 28 cycles.*

*Optimisation 2 : Pragma HLS unroll – on a plus d'itérations et ça demande au total 15 cycles donc on a le maximum de parallélisme. Notez que ça se voit aussi dans l'utilisation des ressources (44 DSPs).*

*Optimisation 3 : Pragma HLS pipeline + Pragma HLS ARRAY_PARTITION shif_reg car on a maintenant 1 écriture. – on a donc 11 itérations, une latence de 6, un II = 1. Ça veut donc dire 6 + 11 = 17 cycles. On prend le même nombre de DSP que l'optimisation 1 mais un peu plus de registres. Ce qui est un bon compromis.*

## Question 2 Synthèse de haut niveau

a) En classe, j'ai souvent dit que les architectures RISC de type superscalaire, ou encore VLIW, étaient utiles pour une parallélisation à gros grains alors que le FPGA était utile pour une parallélisation à grains fins. Expliquez dans vos mots ce que cela signifie.

RISC et VLIW utile pour paralléliser au niveau des tâches alors que FPGA est utilse pour paralléliser au niveau des instructions.

b) Expliquez le rôle du code suivant (Figure 2.1) et de sa transformation avec HLS Vivado (Figure 2.2). Expliquez également l'efficacité de ce pragma, c'est-à-dire obtient-on les résultats escomptés à chaque fois?

```
for(int i = 0; i < X; i++) {
  pragma HLS unroll factor=2
  a[i] = b[i] + c[i];
}
```

Figure 2.1

```
for(int i = 0; i < X; i += 2) {
  a[i] = b[i] + c[i];
  if (i+1 >= X) break;
  a[i+1] = b[i+1] + c[i+1];
}
```

Figure 2.2

*Ici on déroule 2 itérations consécutives ça coupe le nombre de deux. Efficace s'il n'y a pas de dépendances de données (e.g. LAE).*